implementation_demo

May 6, 2025

1 LM Watermarking implementation and Demo

This notebook is heavily based on the work here https://github.com/jthickstun/watermark/

[1]:
```
# Notebook parameters (modify as needed)#
#model_name = "facebook/opt-2.7b"
model_name = "huggyllama/llama-7b"
prompt = "Steganography is"
m = 100             # length of generation
n = 256            # watermark sequence length
key = 42           # watermark key
seed = 1           # random seed for reproducibility
```

[2]:
```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

device = torch.device(
    'cuda' if torch.cuda.is_available() else
    ('mps' if torch.backends.mps.is_available() else 'cpu')
)
print(device)
```

```
cpu
```

[3]:
```
# Seed for reproducibility
torch.manual_seed(seed)

# Initialize tokenizer and model
tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    device_map="auto",
)

# Prepare inputs
inputs = tokenizer.encode(prompt, return_tensors='pt').to(device)
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94:
```

```
UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab
(https://huggingface.co/settings/tokens), set it as secret in your Google Colab
and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access
public models or datasets.
  warnings.warn(
You are using the default legacy behaviour of the <class
'transformers.models.llama.tokenization_llama_fast.LlamaTokenizerFast'>. This is
expected, and simply means that the `legacy` (previous) behavior will be used so
nothing changes for you. If you want to use the new behaviour, set
`legacy=False`. This should only be set if you understand what it means, and
thoroughly read the reason why this was added as explained in
https://github.com/huggingface/transformers/pull/24565 - if you loaded a llama
tokenizer from a GGUF file you can ignore this message.

Loading checkpoint shards:   0%|          | 0/2 [00:00<?, ?it/s]
```

[4]:
```python
# Generate normally
outputs = model.generate(
    inputs,
    max_new_tokens=m,
    do_sample=True,
)

# Decode and display
normal_text = tokenizer.decode(outputs[0], skip_special_tokens=True)

print("--- Normal Output ---")
print(normal_text)
```

```
--- Normal Output ---
Steganography is a technique used to hide secret messages within an image, a
song, or other media that are not usually used to convey secure information.
Steganography can be used to conceal the fact that there is a secret message in
the first place, but it can also be used to transmit secret messages. There are
many different methods of encoding secret messages in an image. Some of the
methods are:
Compression method (also called Spread spectrum)
The key challenge with steganography is embedding
```

[ ]:
```python
# Mersenne Twister based RNG for watermarking
def int_32(x): return int(0xFFFFFFFF & x)

class mersenne_rng:
    def __init__(self, seed=5489):
```

```
        self.state = [0]*624
        self.index = 624
        self.upper_mask = (1<<31)
        self.lower_mask = (1<<31) - 1
        # constants
        self.f, self.m = 1812433253, 397
        self.u, self.s, self.b = 11, 7, 0x9D2C5680
        self.t, self.c, self.l = 15, 0xEFC60000, 18
        # initialize state
        self.state[0] = seed
        for i in range(1, 624):
            prev = self.state[i-1]
            self.state[i] = int_32(self.f * (prev ^ (prev >> 30)) + i)

    def twist(self):
        for i in range(624):
            x = (self.state[i] & self.upper_mask) + (self.state[(i+1)%624] &␣
 ↪self.lower_mask)
            xA = x >> 1
            if x % 2: xA ^= 0x9908b0df
            self.state[i] = self.state[(i + self.m) % 624] ^ xA
        self.index = 0

    def rand(self):
        if self.index >= 624:
            self.twist()
        y = self.state[self.index]
        y ^= (y >> self.u)
        y ^= ((y << self.s) & self.b)
        y ^= ((y << self.t) & self.c)
        y ^= (y >> self.l)
        self.index += 1
        return (int_32(y) * (1.0 / 4294967296.0))

# Exponential watermark sampling
def exp_sampling(probs, u):
    # probs: tensor [vocab], u: numpy array [vocab]
    return torch.argmax(torch.from_numpy(u) ** (1.0 / probs.cpu().numpy()),␣
 ↪dim=1).unsqueeze(-1)

# Watermark generation
import numpy as np
import tqdm

def generate_shift(model, prompt_ids, vocab_size, n, m, key):
    rng = mersenne_rng(key)
    # create watermark matrix xi
```

```
    xi = np.array([rng.rand() for _ in range(n * vocab_size)], dtype=np.
↪float32).reshape(n, vocab_size)
    shift = torch.randint(0, n, (1,)).item()

    inputs = prompt_ids.to(model.device)
    attn = torch.ones_like(inputs)
    past = None
    for i in tqdm.tqdm(range(m)):
        with torch.no_grad():
            if past:
                out = model(inputs[:, -1:], past_key_values=past,␣
↪attention_mask=attn)
            else:
                out = model(inputs)
        probs = torch.nn.functional.softmax(out.logits[:, -1, :vocab_size],␣
↪dim=-1)
        u = xi[(shift + i) % n]
        token = exp_sampling(probs, u)
        inputs = torch.cat([inputs, token.to(model.device)], dim=-1)
        past = out.past_key_values
        attn = torch.cat([attn, torch.ones((attn.shape[0],1), device=attn.
↪device)], dim=-1)

    return inputs.cpu()
```

[6]:
```
# Encode prompt & run watermark generation
tokens = tokenizer.encode(prompt, return_tensors='pt')
wm_tokens = generate_shift(model, tokens, len(tokenizer), n, m, key)[0]
watermarked_text = tokenizer.decode(wm_tokens, skip_special_tokens=True)
print()
print("--- Watermarked Output ---")
print(watermarked_text)
```

```
  0%|          | 0/100 [00:00<?, ?it/s]<ipython-input-5-6c860dbcbbc3>:42:
DeprecationWarning: __array_wrap__ must accept context and return_scalar
arguments (positionally) in the future. (Deprecated NumPy 2.0)
  return torch.argmax(torch.from_numpy(u) ** (1.0 / probs.cpu().numpy()),
dim=1).unsqueeze(-1)
100%|     | 100/100 [02:01<00:00,  1.21s/it]


--- Watermarked Output ---
Steganography is the embedding of a file within another file, so that only
someone with the correct knowledge can see it. It is the opposite of encryption.
The term comes from      (stefano) and   ó (grapho) and means "to cover
with wax", because ancient Greeks hid their secret messages inside other
documents by rubbing the ink. Stfano is the Greek adjective.
Any content you embed within another
```

[7]:
```
from numba import njit, prange
import numpy as np
import math

@njit
def levenshtein(x, y, gamma=0.0):
    n = len(x)
    m = len(y)
    A = np.zeros((n + 1, m + 1), dtype=np.float64)
    for i in range(n + 1):
        for j in range(m + 1):
            if i == 0:
                A[i, j] = j * gamma
            elif j == 0:
                A[i, j] = i * gamma
            else:
                cost_sub = math.log(1.0 - y[j - 1, x[i - 1]])
                # deletion
                best = A[i - 1, j] + gamma
                # insertion
                tmp = A[i, j - 1] + gamma
                if tmp < best:
                    best = tmp
                # substitution
                tmp = A[i - 1, j - 1] + cost_sub
                if tmp < best:
                    best = tmp
                A[i, j] = best
    return A[n, m]
```

[8]:
```
import numpy as np
import time
import tqdm

from numba import njit, prange


@njit
def detect(tokens, n, k, xi, gamma=0.0):
    m = len(tokens)
    n = len(xi)

    A = np.empty((m - (k - 1), n))
    for i in range(m - (k - 1)):
        for j in range(n):
```

```
            A[i][j] = levenshtein(tokens[i : i + k], xi[(j + np.arange(k)) %␣
 ↪n], gamma)

    return np.min(A)

def permutation_test(tokens, key, n, k, vocab_size, n_runs=100):
    rng = mersenne_rng(key)
    xi = np.array(
        [rng.rand() for _ in range(n * vocab_size)], dtype=np.float32
    ).reshape(n, vocab_size)
    test_result = detect(tokens, n, k, xi)

    p_val = 0
    for run in tqdm.tqdm(range(n_runs)):
        xi_alternative = np.random.rand(n, vocab_size).astype(np.float32)
        null_result = detect(tokens, n, k, xi_alternative)

        # assuming lower test values indicate presence of watermark
        p_val += null_result <= test_result

    return (p_val + 1.0) / (n_runs + 1.0)


# Run decoding test
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokens = tokenizer.encode(watermarked_text, return_tensors='pt',␣
 ↪truncation=True, max_length=2048).numpy()[0]
tok_list = tokens.tolist()
print(tok_list)
start = time.time()
p_val_watermarked = permutation_test(tok_list, key, n, len(tok_list),␣
 ↪len(tokenizer))
print()
print(f"p-value: {p_val_watermarked:.4f} (elapsed {time.time()-start:.1f}s)")
print(f"Is watermarked: {p_val_watermarked<0.01}")
```

```
[1, 2443, 6249, 5275, 338, 278, 23655, 310, 263, 934, 2629, 1790, 934, 29892,
577, 393, 871, 4856, 411, 278, 1959, 7134, 508, 1074, 372, 29889, 739, 338, 278,
11564, 310, 20956, 29889, 13, 1576, 1840, 5304, 515, 29871, 30164, 30137, 30151,
30237, 30110, 30133, 30206, 313, 303, 1389, 1562, 29897, 322, 29871, 30197,
30147, 30110, 30237, 29980, 313, 4262, 29877, 29897, 322, 2794, 1346, 517, 4612,
411, 281, 1165, 9363, 1363, 12297, 18438, 2039, 20552, 1009, 7035, 7191, 2768,
916, 10701, 491, 14051, 10549, 278, 297, 29895, 29889, 624, 29888, 1562, 338,
278, 12311, 594, 25674, 29889, 13, 10773, 2793, 366, 8297, 2629, 1790]

100%|      | 100/100 [00:26<00:00,  3.76it/s]
```

```
p-value: 0.0099 (elapsed 35.8s)
Is watermarked: True
```

[9]:
```
!pip install googletrans==4.0.0-rc1
from googletrans import Translator
import random

def back_translate(text, target_language='es'):
    """Translates text to a target language and back to the original language.
    """
    translator = Translator()
    translated = translator.translate(text, dest=target_language).text
    back_translated = translator.translate(translated, dest='en').text
    return back_translated

#def add_noise(text, noise_level=0.1):
#    """Adds random noise to the text."""
#    words = text.split()
#    num_noise_words = int(len(words) * noise_level)
#    noise_words = ["noise" for _ in range(num_noise_words)]
#    for i in range(num_noise_words):
#        words.insert(random.randint(0, len(words)), noise_words[i])
#    return " ".join(words)

# Apply back-translation to the watermarked text
back_translated_text = back_translate(back_translate(watermarked_text, "fr"),␣
    "ru")
print("Back translated: ", back_translated_text)

# Encode the back-translated text
back_translated_tokens = tokenizer.encode(back_translated_text,␣
    return_tensors='pt', truncation=True, max_length=2048).numpy()[0]
back_translated_tok_list = back_translated_tokens.tolist()

# Detect watermark in the back-translated text
start = time.time()
p_val_back_translated = permutation_test(back_translated_tok_list, key, n,␣
    len(back_translated_tok_list), len(tokenizer))
print()
print(f"p-value (back-translated): {p_val_back_translated:.4f} (elapsed {time.
    time()-start:.1f}s)")
print(f"Is watermarked (back-translated): {p_val_back_translated<0.01}")
```

```
Requirement already satisfied: googletrans==4.0.0-rc1 in
/usr/local/lib/python3.11/dist-packages (4.0.0rc1)
Requirement already satisfied: httpx==0.13.3 in /usr/local/lib/python3.11/dist-
```

```
packages (from googletrans==4.0.0-rc1) (0.13.3)
Requirement already satisfied: certifi in /usr/local/lib/python3.11/dist-
packages (from httpx==0.13.3->googletrans==4.0.0-rc1) (2025.1.31)
Requirement already satisfied: hstspreload in /usr/local/lib/python3.11/dist-
packages (from httpx==0.13.3->googletrans==4.0.0-rc1) (2025.1.1)
Requirement already satisfied: sniffio in /usr/local/lib/python3.11/dist-
packages (from httpx==0.13.3->googletrans==4.0.0-rc1) (1.3.1)
Requirement already satisfied: chardet==3.* in /usr/local/lib/python3.11/dist-
packages (from httpx==0.13.3->googletrans==4.0.0-rc1) (3.0.4)
Requirement already satisfied: idna==2.* in /usr/local/lib/python3.11/dist-
packages (from httpx==0.13.3->googletrans==4.0.0-rc1) (2.10)
Requirement already satisfied: rfc3986<2,>=1.3 in
/usr/local/lib/python3.11/dist-packages (from
httpx==0.13.3->googletrans==4.0.0-rc1) (1.5.0)
Requirement already satisfied: httpcore==0.9.* in
/usr/local/lib/python3.11/dist-packages (from
httpx==0.13.3->googletrans==4.0.0-rc1) (0.9.1)
Requirement already satisfied: h11<0.10,>=0.8 in /usr/local/lib/python3.11/dist-
packages (from httpcore==0.9.*->httpx==0.13.3->googletrans==4.0.0-rc1) (0.9.0)
Requirement already satisfied: h2==3.* in /usr/local/lib/python3.11/dist-
packages (from httpcore==0.9.*->httpx==0.13.3->googletrans==4.0.0-rc1) (3.2.0)
Requirement already satisfied: hyperframe<6,>=5.2.0 in
/usr/local/lib/python3.11/dist-packages (from
h2==3.*->httpcore==0.9.*->httpx==0.13.3->googletrans==4.0.0-rc1) (5.2.0)
Requirement already satisfied: hpack<4,>=3.0 in /usr/local/lib/python3.11/dist-
packages (from h2==3.*->httpcore==0.9.*->httpx==0.13.3->googletrans==4.0.0-rc1)
(3.0.0)
Back translated:  Steganography is the inclusion of a file in another file, so
only someone who has the right knowledge could see this. This is the opposite of
encryption.
The term comes from      (stefano) and   ó (grapho) and means "wax cover"
because the ancient Greeks hid their secret messages within other documents,
rubbing the ink.The camp is a Greek adjective.
Any content that you integrate into another

100%|      | 100/100 [00:26<00:00,  3.80it/s]


p-value (back-translated): 0.0099 (elapsed 33.4s)
Is watermarked (back-translated): True
```

[20]:
```python
tokens_control = tokenizer.encode(normal_text, return_tensors='pt',␣
 ↪truncation=True, max_length=2048).numpy()[0]
tok_control_list = tokens_control.tolist()
print(tok_control_list)
start = time.time()
```

```
p_val_control = permutation_test(tok_control_list, key, n,␣
  ↪len(tok_control_list), len(tokenizer))
print()
print(f"p-value (control): {p_val_control:.4f} (elapsed {time.time()-start:.
  ↪1f}s)")
print(f"Is watermarked (control): {p_val_control<0.01}")
```

```
[1, 2443, 6249, 5275, 338, 263, 11043, 1304, 304, 9563, 7035, 7191, 2629, 385,
1967, 29892, 263, 4823, 29892, 470, 916, 5745, 393, 526, 451, 5491, 1304, 304,
27769, 11592, 2472, 29889, 2443, 6249, 5275, 508, 367, 1304, 304, 10628, 284,
278, 2114, 393, 727, 338, 263, 7035, 2643, 297, 278, 937, 2058, 29892, 541, 372,
508, 884, 367, 1304, 304, 22649, 7035, 7191, 29889, 1670, 526, 1784, 1422, 3519,
310, 8025, 7035, 7191, 297, 385, 1967, 29889, 3834, 310, 278, 3519, 526, 29901,
13, 1523, 2590, 1158, 313, 15189, 2000, 1706, 949, 18272, 29897, 13, 1576, 1820,
18766, 411, 1886, 6249, 5275, 338, 23655]
```

```
100%|      | 100/100 [00:27<00:00,  3.68it/s]


p-value (control): 0.7228 (elapsed 34.7s)
Is watermarked (control): False
```

[21]:
```
print(f"""
--- Plain/Control text (p={p_val_control:.4f}) ---
{normal_text}

--- Watermarked (p={p_val_watermarked:.4f}) ---
{watermarked_text}

--- Edited (p={p_val_back_translated:.4f}) ---
{back_translated_text}
""")
```

```
--- Plain/Control text (p=0.7228) ---
Steganography is a technique used to hide secret messages within an image, a
song, or other media that are not usually used to convey secure information.
Steganography can be used to conceal the fact that there is a secret message in
the first place, but it can also be used to transmit secret messages. There are
many different methods of encoding secret messages in an image. Some of the
methods are:
Compression method (also called Spread spectrum)
The key challenge with steganography is embedding

--- Watermarked (p=0.0099) ---
Steganography is the embedding of a file within another file, so that only
someone with the correct knowledge can see it. It is the opposite of encryption.
The term comes from      (stefano) and   ó (grapho) and means "to cover
```

with wax", because ancient Greeks hid their secret messages inside other documents by rubbing the ink. Stfano is the Greek adjective.
Any content you embed within another

--- Edited (p=0.0099) ---
Steganography is the inclusion of a file in another file, so only someone who has the right knowledge could see this. This is the opposite of encryption. The term comes from (stefano) and ó (grapho) and means "wax cover" because the ancient Greeks hid their secret messages within other documents, rubbing the ink.The camp is a Greek adjective.
Any content that you integrate into another